PROJECT GUIDE · PART 2

StreetCare AI Concepts Handbook

Every important idea used in the project, in plain words: Python design, exact data cleaning, a secured service with roles, machine learning, NLP, LLM safety, RAG over council documents and a LangGraph workflow with a human gate. Each comes with a definition, an example, where it lives in the code, and the questions interviewers ask.

71 concepts | 9 parts | ~106 interview questions | figures from the specification & config | diagrams & charts

---

Companion to *StreetCare AI · Architecture & Workflow*
Prepared for the StreetCare AI team

How to use this handbook

Each concept follows the same structure, so you can revise fast before a review or an interview.

📘 **Definition**
One or two sentences you can say word for word.

💬 **In simple words / analogy**
How to explain it to a non-expert, or to check you really understand it.

⚙️ **How it works + example**
The mechanics, a diagram or chart, and a small code or number example.

📍 **Where it is used in StreetCare AI**
The exact module and setting from the specification: your proof of hands-on understanding.

INTERVIEW TIP

Answer in three beats: **what it is** (definition) → **why it matters** (the problem it solves) → **where I used it** (StreetCare + a figure). For example: *"Data leakage is information from evaluation or the future reaching training... it makes scores lie... in StreetCare every imputer, scaler and TF-IDF vocabulary is fitted inside the pipeline on the train file only, and the model must still reach a held-out ROC AUC of 0.80."*

A NOTE ON NUMBERS

The code is still being built. Figures here are the specification's expected outputs (row counts, document and section counts) and the settings in `config.py` (thresholds, minimums, model settings). Model scores will come from `run_model_training.py`; this handbook never invents them.

Part 1–3
Python · Data · Service & security

Part 4–5
Machine learning · NLP

Part 6–9
LLMs · RAG · Agents · Testing

Contents

Part 1: Python & software design

01 Modules, packages & layered architecture
02 Configuration as a single source of truth
03 Pure functions & never mutating inputs
04 Refuse, never guess: errors as a design tool
05 Regular expressions for validation & parsing
06 Exact decimal arithmetic & rounding half up
07 Type hints, TypedDict & dataclass-like state
08 Idempotent scripts & entry points

Part 2: Data engineering

09 Data cleaning & missing-value tokens
10 Canonical values & value mappings
11 Unit conversion as a reading
12 Deduplication & keeping the latest revision
13 Data quality profiling
14 pandas dtypes: nullable integers & text
15 Feature engineering from business rules

Part 3: Web service, security & storage

16 REST APIs & HTTP status codes
17 FastAPI routers & dependency injection
18 Pydantic validation & plain-value replies
19 Password hashing with PBKDF2 & salts
20 Session tokens
21 Role-based access control (RBAC)
22 Separation of duties & district authorisation
23 Concurrency: locks & conflicts
24 SQL, SQLite & the SQLAlchemy ORM
25 Transactions & atomic writes
26 Thin clients: Streamlit over an API

Part 5: Natural language processing (NLP)

42 Text cleaning: markup, entities, whitespace, capitals
43 Sentence splitting
44 Tokenisation, n-grams & TF-IDF
45 Text classification: the nine sentence types
46 Rule-based information extraction
47 Comparing evidence with the record: disagreement flags
48 Attention & transformers, in brief

Part 6: Large language models & GenAI safety

49 Large language models (LLMs)
50 Prompt engineering & labelled-field output
51 Validating LLM output & refusing bad replies
52 Token & latency accounting
53 Grounding: facts in, prose out
54 Output guardrails for documents sent out
55 Privacy rules & role-aware answers

Part 7: Retrieval & the knowledge base (RAG)

56 RAG: the big idea
57 PDF extraction & section splitting
58 Chunking with overlap
59 Sentence embeddings
60 Vector databases & cosine similarity (Chroma)
61 Exact section lookup vs similarity search
62 Relevance thresholds, sources & refusals
63 Reading rules from documents

Part 8: Agents, LangGraph & human-in-the-loop

64 Agents as single LLM calls
65 LangGraph: state, nodes, edges & reducers

Part 4: Machine learning

27 Supervised learning: classification & regression
28 Held-out evaluation & train/eval files
29 Data leakage
30 Preprocessing: imputation, scaling, one-hot
31 scikit-learn Pipeline & ColumnTransformer
32 Logistic regression
33 Random forest regression
34 ROC curve & AUC
35 R^2 and MAE for regression
36 Macro F1 for multi-class text
37 Overfitting & the train vs held-out gap
38 Behavioural model tests (monotonic checks)
39 Model gating by minimum score
40 Model persistence & feature schemas
41 From probability to band, and estimates that inform

66 Conditional routing
67 Human-in-the-loop with interrupt
68 Checkpointing & resuming after a restart
69 Idempotent resume & failure recovery

Part 9: Testing & engineering practice

70 Automated testing with pytest
71 Secrets, configuration & reproducibility

Extras

★ Rapid-fire interview bank
★ Explain the project: 30 s & 2 min
★ Numbers to remember

PART 1

Python & software design

The habits that make a large specification buildable, testable and exact.

01 Modules, packages & layered architecture

Python

> **Definition.** A **module** is one `.py` file; a **package** is a folder of modules. A **layered architecture** arranges packages so that each one only depends on the layers beneath it.

IN SIMPLE WORDS

StreetCare has eleven folders, each with one job: cleaning, models, documents, tools, storage, agents, chat, access, API, dashboard. A lower layer never imports a higher one, so you can build and test from the bottom up.

HOW IT WORKS

- **config** at the bottom, depending on nothing.
- **data_pipeline** → **modeling** → **tools**; **knowledge** → **tools**; **persistence** beside them.
- **agentic** and **chat** use tools and storage; **access + api** sit on top.
- The **dashboard** imports nothing from the project: it only calls the API over HTTP.

> **Where it is used in StreetCare AI:** The whole `Project/` tree: 11 packages and 6 `run_*.py` entry points. The specification fixes every module path and function name, because a marking suite imports them.

INTERVIEW QUESTIONS

Q. Why can't a lower layer import a higher one?

A. It creates import cycles and hidden coupling. If `tools` imported `api`, you could not test screening without starting a web service.

Q. Why does the dashboard not import the tools directly?

A. Then the rules would run in two places and could drift. Going through the API means one implementation, and the API enforces access.

02 Configuration as a single source of truth

Python

> **Definition.** Keeping every path, mapping, pattern, threshold, role and message in **one module** that the rest of the code imports, instead of retyping values.

IN SIMPLE WORDS

If the overrun ratio, a spelling or a refusal message ever changes, it changes in `config.py` and everything follows.

HOW IT WORKS

- Paths: `RAW_*_FILE`, `CLEAN_*_FILE`, `*_MODEL_FILE`.
- Vocabularies: `DEFECT_TYPE_MAPPING`, `STREET_NAME_MAPPING`, `FORM_VALUE_MAPPINGS`.
- Rules and limits: `OVERRUN_RATIO = 1.3`, `HIGH_COLUMN_TALLER_THAN_METRES = 8`, `GROUP_SIZE_MINIMUM = 5`.
- **Not** in config: any requirement or its figure. Those exist only in the defect standards.

EXAMPLE

```
from config import config

if expected > config.OVERRUN_RATIO * planned:      # never 1.3 typed by hand
    text = config.OVERRUN_TEMPLATE.format(expected_crew_hours=expected, planned_crew_hours=planned)
```

> **Where it is used in StreetCare AI:** `config/config.py` (1,393 lines, supplied complete). Messages such as `DECIDING_ROLE_MESSAGE` are used word for word, because tests read them.

INTERVIEW QUESTIONS

Q. Why not put the requirement limits in config too?

A. Because the specification makes the PDFs the only source of rules. Reading them at run time means a standard can change without a code change, and the carriageway and footway limits can never be swapped by a copy-paste.

Q. What goes in config versus `.env`?

A. Config holds non-secret settings that belong in the repository. `.env` holds secrets (the Gemini key) and is never committed.

03 Pure functions & never mutating inputs

Python Data

> **Definition.** A **pure function** returns a result computed only from its arguments and changes nothing else, not even the objects it was given.

IN SIMPLE WORDS

Every cleaning step hands back a *new* table and leaves the one it received exactly as it was.

HOW IT WORKS

- Start with `df = dataframe.copy()`, change the copy, return it.
- Same input → same output, so each step is trivial to test.
- Screening is pure too: one report + its standards → one screening dict.
- Side effects (files, databases) live only at the edges: loaders, savers, repository.

EXAMPLE

```
def remove_duplicate_records(dataframe):
    result = dataframe.drop_duplicates(keep="first")
    return result.reset_index(drop=True)      # numbered 0, 1, 2 … no gaps
```

Where it is used in StreetCare AI: `data_pipeline/common_cleaning.py` : "every function here returns a repaired table or column and leaves the one it was given untouched; none reads or writes a file."

INTERVIEW QUESTIONS

Q. Why does mutating an input cause bugs in pandas?

A. The caller may still use the original. A step that quietly changes it makes a later step, or a profile of the raw data, see the wrong values.

Q. How do pure functions help testing?

A. No setup, no files, no mocks: call it with a small DataFrame and assert on the result.

04 Refuse, never guess: errors as a design tool

Python

Definition. Raising an exception with a **precise message** the moment data does not match the rules, instead of guessing a value or silently dropping it.

IN SIMPLE WORDS

If a depth is written `4.5 cms` and the rules don't list `cms`, the run stops and says so. Someone fixes it once at the source, and nothing is ever screened on a guess.

HOW IT WORKS

- `ValueError` naming the **column** and the **values** it could not read.
- Refusals at the gate use the configuration's own words (e.g. `OWN_REVIEW_MESSAGE`).
- The API turns validation failures into 422 and decision conflicts into 409.
- A model below its minimum stops training and nothing is saved.

EXAMPLE

```
bad = column[~column.isna() & ~column.str.fullmatch(pattern)]
if not bad.empty:
    raise ValueError(f"{name}: cannot read {sorted(bad.unique())}")
```

Where it is used in StreetCare AI: Every `common_cleaning` function; `find_decision_refusal` ; `parse_labelled_response` ; the service start-up check.

INTERVIEW QUESTIONS

Q. Isn't it friendlier to guess?

A. A guessed unit can pass or fail the wrong report, and nobody notices. A loud, specific error is fixed once.

Q. What makes a good error message?

A. It names what was wrong (column, value, rule) so the reader can act without opening the code.

05 Regular expressions for validation & parsing

Python NLP

> **Definition.** A compact pattern language to **match, capture or split** text. StreetCare uses them to read identifiers, find section headings, split sentences and pull figures out of statements.

HOW IT WORKS

- `^` and `$` (or `fullmatch`) anchor the whole value.
- Groups `( )` capture the part you need, e.g. the number of a reference.
- Look-arounds `(?=…)` `(?⩽…)` check context without consuming it.
- `(?i)` ignores case; `(?m)` makes `^` match at every line.

EXAMPLE

Pattern (from config)	Meaning	Matches
`^(?:SC(?=[0-9]{7}$)\|SC-)?([0-9]{1,7})$`	any of the three channel notations; capture the number	SC0050118 · SC-50118 · 50118
`^SWP[0-9]{7}$`	a street works permit	SWP0071240
`(?m)^([0-9]{1,2})\. [A-Z]`	a numbered section heading	4. Requirements
`(?i)([0-9]+(?:\.[0-9]+)?) m clear`	a clear height in a statement	"only 2.1 m clear"
`(?⩽[.!?])\s+`	split statements into sentences	space after . ! ?

> **Where it is used in StreetCare AI:** `config.py` holds every pattern; `common_cleaning`, `chunk_splitting`, `standard_reading`, `evidence_tools` and the chat's reference finder use them.

INTERVIEW QUESTIONS

Q. Why the look-ahead `SC(?=[0-9]{7}$)` ?

A. `SC` without a dash is only valid when exactly seven digits follow; the look-ahead checks that without consuming the digits, so the group still captures them.

Q. Greedy vs lazy?

A. `.*` takes as much as possible; `.*?` as little. The requirements-section pattern uses a lazy match so it stops at the first closing phrase.

06 Exact decimal arithmetic & rounding half up

Python Data

> **Definition.** `decimal.Decimal` stores numbers in base ten exactly, so `1.75 × 25.4` is exactly `44.45`. **Round half up** sends an exact half away from zero, and it is applied **once**, at the reading's own step.

IN SIMPLE WORDS

Floats store most decimals approximately: `0.1 + 0.2 = 0.30000000000000004`. When a requirement is met or missed by the last digit, "approximately" is not good enough.

HOW IT WORKS

- Build from text: `Decimal("4.5")`, never `Decimal(4.5)`.
- Convert with the configured multiplier, e.g. `MILLIMETRES_PER_INCH = Decimal("25.4")`.
- Round **once**, at the reading's own step: `.quantize(Decimal("1"), ROUND_HALF_UP)` for mm, `Decimal("0.1")` for metres and hours.
- Compare against limits that are also `Decimal`, read from the standard.

EXAMPLE

```
from decimal import Decimal, ROUND_HALF_UP
Decimal("1.25") * Decimal("25.4")                  # 31.750 exactly
(Decimal("1.25") * Decimal("25.4")).quantize(Decimal("1"), ROUND_HALF_UP)   # 32 mm
(Decimal("45") / Decimal("60")).quantize(Decimal("0.1"), ROUND_HALF_UP)    # 0.8 hours
```

Where it is used in StreetCare AI: `convert_readings` (4.5 cm → 45 mm, 1.75 in → 44 mm, 4 ft → 1.2 m, 45 min → 0.8 h); likelihood rounded to 0.01; crew hours to 0.1; disagreement checks compare unrounded Decimals.

INTERVIEW QUESTIONS

Q. Why not Python's built-in `round()`?

A. It uses banker's rounding (half to even) on binary floats: `round(2.5) == 2`. The Report Rules say half up.

Q. Why round only once?

A. Rounding twice can change the answer: 0.449 → 0.45 → 0.5, whereas once gives 0.4.

07 Type hints, TypedDict & dataclass-like state

Python

Definition. Type hints document what a function takes and returns. A **TypedDict** declares the exact keys (and their types) a dictionary must have.

IN SIMPLE WORDS

The review workflow passes one dictionary from node to node. `ReportReviewState` lists every key it may hold, and nothing else.

HOW IT WORKS

- Readers (and editors) see the shape at a glance.
- LangGraph reads the TypedDict to know the state's keys.
- `Annotated[int, operator.add]` attaches a **reducer**: values returned by nodes are added, not replaced.
- Pydantic uses the same hints to validate API bodies.

EXAMPLE

```
class ReportReviewState(TypedDict):
    report_reference: str
    coordinator_request: str
    screening: dict
    evidence_findings: list[dict]
    review_decision: dict
    llm_call_count: Annotated[int, operator.add]     # each agent node returns 1
```

Where it is used in StreetCare AI: `agentic/state.py` ("every key above is declared, and nothing else is"); `api/schemas.py`.

INTERVIEW QUESTIONS

Q. Are type hints checked at run time?

A. Not by Python itself. Tools such as mypy check them statically; Pydantic and LangGraph read them at run time for validation and state handling.

08 Idempotent scripts & entry points

Python

Definition. An **idempotent** operation gives the same result however many times it runs. An **entry point** is a script with a `main()` that runs only when executed, never when imported.

HOW IT WORKS

- `run_database_setup.py` creates 16 accounts the first time and "created: 0, already present: 16" the second.
- `if __name__ == "__main__": sys.exit(main())`: importing the module starts nothing.
- Reports are written with sorted keys, so rerunning produces the identical file.
- Indexing rebuilds the collection from scratch, so reruns don't duplicate passages.

EXAMPLE

```
def main():
    created = present = 0
    for account in config.SEED_USER_ACCOUNTS:
        if fetch_user_account(session, ...) is None:
            insert_user_account(session, account); created += 1
        else:
            present += 1
    print(f"accounts created: {created}, already present: {present}")

if __name__ == "__main__":
    sys.exit(main())
```

Where it is used in StreetCare AI: The six `run_*.py` entry points; "Running the file makes main's result the process's exit status; importing the module starts nothing."

INTERVIEW QUESTIONS

Q. Why does idempotency matter for setup scripts?

A. Ops people rerun things. A non-idempotent setup would create duplicate users or crash on the second run.

PART 2

Data engineering

Turning five messy extracts into tables a model and a reviewer can trust.

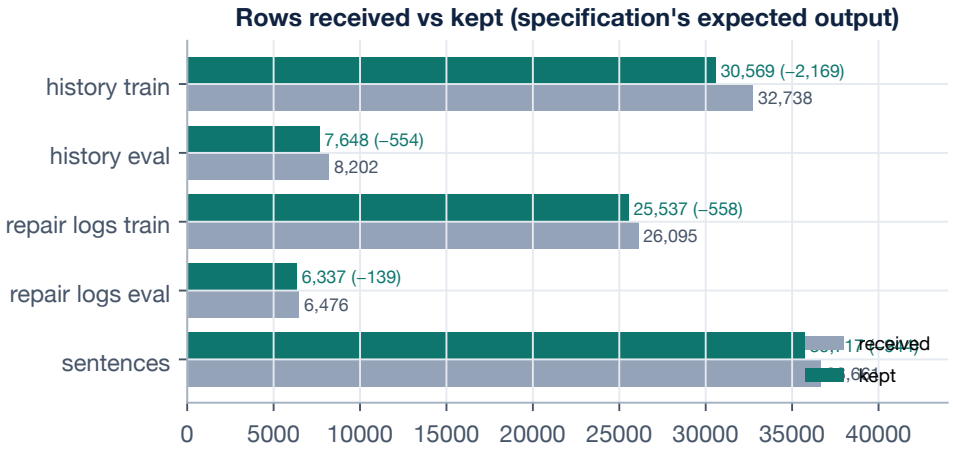
09 Data cleaning & missing-value tokens

Data

Definition. Finding and repairing (or refusing) values that are duplicated, blank, impossible or written in the wrong form, **before** anything learns from them or is screened.

HOW IT WORKS

- Read raw files with **every column as text** (`dtype=str` , `keep_default_na=False`), so pandas guesses nothing.
- Strip whitespace; a value matching `NULL_TOKENS` in any case becomes missing: `""` , `n/a` , `na` , `-` , `null` , `not known` , `caller unsure` .
- Then duplicates, identifiers, references, spellings, numbers, readings, limits, revisions, targets, in that order.
- Some gaps are **facts**, not defects: a water depth on a footway report stays empty.



Received vs kept for each training extract (specification's expected output)

Where it is used in StreetCare AI: `data_pipeline/dataset_loader.py` and `common_cleaning.py` ; e.g. history train 32,738 → 30,569; live queue 45 → 40; register 29 → 28 (23 live).

INTERVIEW QUESTIONS

Q. Why read everything as text first?

A. pandas would turn `"N/A"`, `"0050118"` or `"2 in"` into NaN, an integer or an object column by guessing. Reading text means every repair is explicit and testable.

Q. Drop or keep a row with a missing feature?

A. Keep it: the model pipeline imputes features. Only a missing *target* (no answer) removes a training row, because there is nothing to learn.

10 Canonical values & value mappings

Data

> **Definition.** Folding every spelling a channel writes onto **one canonical value** using an explicit dictionary, matched without regard to case.

HOW IT WORKS

- Lower-case the key and the value before looking up.
- A value **not** in the mapping is refused, naming column and values.
- A missing value stays missing; columns not named are left alone.
- Separate mappings per dataset: `FORM_`, `HISTORY_`, `LOG_`, `SENTENCE_`, `DOCUMENT_`, `REGISTER_VALUE_MAPPINGS`.

EXAMPLE

Column	As written	Canonical
defect_type	CW · CARRIAGEWAY · Pothole or road surface	Carriageway
linked_defect_type	NIL · None · (blank meaning none)	No linked defect
street_category	TS · traffic sensitive · Main route	Traffic sensitive
location_fixed	Y · true · Yes	Yes
street_name	top road · TOP ROAD	Top Road

> **Where it is used in StreetCare AI:** `normalize_category_values(dataframe, value_mappings)`; the profiler counts `category_values_not_written_canonically` as evidence the step was needed.

INTERVIEW QUESTIONS

Q. Why not fuzzy-match unknown spellings?

A. A fuzzy match could turn `FW` into Footway when the caller meant something else. The rules list every accepted spelling; anything else is a data problem to fix.

11 Unit conversion as a reading

Data

> **Definition.** A **reading** describes how one measurement column is understood: which notations it may arrive in, the multiplier or divisor of each, the step to round to and the final dtype.

HOW IT WORKS

- Match the value against each notation's pattern; capture the number.
- Multiply or divide as a Decimal; round half up once to the step.
- Then `treat_values_below_minimum_as_missing` / `…above_maximum…` blank impossible readings, never correcting them.

EXAMPLE

Reading	Unit	Notations	Example
depth, width, standing water	whole mm	mm · cm (×10) · in (×25.4)	4.5 cm → 45 · 1.75 in → 44
reach, clear height, column height	m, 0.1	m · ft (×0.3048)	4 ft → 1.2
column lean	degrees, 0.1	degrees	0–90 kept
crew hours	hours, 0.1	hours · hrs · min (÷60)	45 min → 0.8 · 210 min → 3.5

> **Where it is used in StreetCare AI:** `MILLIMETRE_READING`, `METRE_READING`, `DEGREE_READING`, `CREW_HOURS_READING` in config; `convert_readings(dataframe, readings)`.

INTERVIEW QUESTIONS

Q. Why blank an impossible value rather than clip it?

A. Clipping invents a reading nobody took. Blanking makes the requirement "Not documented", which sends the report for inspection: the safe outcome.

12 Deduplication & keeping the latest revision

Data

> **Definition.** Removing rows that repeat exactly, then keeping each report only at its **highest revision number**, wherever its rows sit in the file.

IN SIMPLE WORDS

A resident sends a report, then sends it again with the depth added. Revision 2 *is* the report; revision 1 must never be screened or learned from.

HOW IT WORKS

- Exact duplicates first (rows identical in every column).
- Then per `report_reference`, keep the row with the max `revision_number`.
- Refuse a row with no reference or revision, or two rows with the same reference *and* revision: the file contradicts itself.
- Keep the surviving rows in their original order; renumber 0, 1, 2…

- In the history, a report whose latest revision has no answer is left out; an earlier revision never stands in.

EXAMPLE

```
latest = df.groupby("report_reference")["revision_number"].transform("max")
kept = df[df["revision_number"] == latest].reset_index(drop=True)
```

Where it is used in StreetCare AI: `keep_latest_revision_of_each_report` : the live queue goes from 45 rows to 40 reports; the profiler reports `answered_earlier_revisions_removed` .

INTERVIEW QUESTIONS

Q. Why not just keep the last row in the file?

A. Channels don't send rows in order. The revision number, not file position, says which is newest.

13 Data quality profiling

Data

Definition. Measuring a dataset **before and after** cleaning, so a reviewer can see that every repair was needed and that nothing was lost unexplained.

HOW IT WORKS

- Profile: rows received / kept / removed, duplicate rows, rows missing the target, columns, missing cells before and after.
- Details: references not canonical, readings by unit as written, values below minimum / above maximum, non-canonical categories, target shares.
- Text details: sentences with markup, encoding artefacts, in capitals, lengths that disagreed, too few words.
- The removal reasons **add up** to `rows_removed` . Written as JSON, indent 2, keys sorted, so reruns are identical.

EXAMPLE

```
{ "report_history_train": { "rows_received": 32738, "rows_kept": 30569, "rows_removed": 2169,
    "duplicate_rows": …, "answered_earlier_revisions_removed": …, "rows_missing_target": …,
    "readings_by_unit_as_written": { "defect_depth_mm": { "cm": …, "in": …, "mm": … } }, … } }
```

Where it is used in StreetCare AI: `quality_profiler.py` → `output/data_quality_report.json` .

INTERVIEW QUESTIONS

Q. Why must the removal reasons add up?

A. It proves no row disappeared without a reason, which is the core of an auditable pipeline.

14 pandas dtypes: nullable integers & text

Data

> **Definition.** A column's **dtype** decides what it can hold. Classic `int64` cannot hold a missing value; pandas' nullable `Int64` can (as `<NA>`).

HOW IT WORKS

- Counts and revisions become `WHOLE_NUMBER_DTYPE = "Int64"`, so a gap stays a gap instead of turning the column into floats.
- Free text is held as `TEXT_DTYPE = "str"`.
- Model inputs: numbers as float with NaN, categories as text with NaN, never `None`, the string `"nan"` or `""`.
- Cleaned files are read back with types inferred, but only an empty cell is missing, so the canonical value `None` (no traffic management) stays text.

> **Where it is used in StreetCare AI:** `convert_whole_numbers`; `load_processed_dataset`; `build_feature_frame`.

INTERVIEW QUESTIONS

Q. Why is `"None"` a problem when reading a CSV?

A. pandas treats the text `None` as missing by default. Here it is a real category value, so the loader must only treat empty cells as missing.

15 Feature engineering from business rules

Data ML

> **Definition.** Creating new input columns from domain knowledge so the model can see a pattern the raw answers hide.

HOW IT WORKS

- **Requirement counts**: screen every past report under the standards read from the PDFs and count Met / Not met / Not documented.
- **Trigger flags**: 1 or 0 for live street works, traffic-sensitive street and high column, worked out exactly as for a live report.
- The same functions produce these columns for training and for live scoring, so they always mean the same thing.

EXAMPLE

New column	How it's made	Why it helps
requirements_not_met_count	screen_report under the report's standards	a defect that failed its standard is likelier to return
requirements_not_documented_count	blank answers the standard needed	uncertainty about the defect
live_street_works_trigger	a live, conflicting permit on the street	works disturb repairs

traffic_sensitive_trigger	busy street + carriageway/drainage type	more wear; longer repairs

Where it is used in StreetCare AI: `add_requirement_counts` and `add_coordination_trigger_flags` in `feature_builder.py`; the repeat model must react to them in the right direction (see behavioural checks).

INTERVIEW QUESTIONS

Q. Isn't reusing screening as a feature a kind of leakage?

A. No: screening uses only answers known when the report arrives, never the outcome. Leakage is using information from after the decision.

PART 3

Web service, security & storage

Serving everything over HTTP to three roles, safely, with a full audit trail.

16 REST APIs & HTTP status codes

API

Definition. REST is a style for web APIs: resources live at URLs (`/reviews/REV-…`) and the HTTP method says what to do with them.

EXAMPLE

Code	Means	In StreetCare
200	OK	every successful GET and the decision
201	Created	`POST /reviews` stored a new review
401	not signed in	no token, or a token no account holds
403	signed in but not allowed	an adviser calling `GET /queue`; an inspector from another district
404	not found	no open report or review with that identifier
409	conflict with current state	review already decided; a different decision in progress; a refused decision
422	body failed validation	a question of 2 characters; an unknown decision

Where it is used in StreetCare AI: `api/routes.py`: 11 routes on a public and a guarded router. "Every endpoint answers 200 when it succeeds, except POST /reviews, which answers 201."

INTERVIEW QUESTIONS

Q. Why is POST /reviews not idempotent?

A. Each call starts a new run and spends five LLM calls. That is why it is guarded, and why a report has one lock.

Q. 401 vs 403?

A. 401: we don't know who you are. 403: we know, and your role doesn't include this permission.

17 FastAPI routers & dependency injection

API

> **Definition.** FastAPI builds routes from typed Python functions. **Dependency injection** (`Depends`) lets a route declare what it needs, such as the signed-in user, and FastAPI supplies it.

HOW IT WORKS

- `public_router` : `/auth/signin` and `/auth/signout` .
- `guarded_router` : every other route, included with `dependencies=[Depends(authorize_request)]` , so the guard cannot be forgotten.
- `SignedInUser` is an `Annotated` alias that injects the account the guard returned.
- Swagger docs at `/docs` come from the code.

EXAMPLE

```
guarded_router = APIRouter(dependencies=[Depends(authorize_request)])

@guarded_router.post("/reviews", status_code=201)
def start_report_review(review_request: ReviewRequest, signed_in_user: SignedInUser, http_request:
Request):
    ...
```

> **Where it is used in StreetCare AI:** `api/routes.py` (`service_application` titled `API_TITLE`); `run_api_service.py` serves it with Uvicorn on 0.0.0.0:8081.

INTERVIEW QUESTIONS

Q. Why put auth on the router, not in each handler?

A. One place to get right. A new endpoint added to the guarded router is protected automatically.

18 Pydantic validation & plain-value replies

API Python

> **Definition.** Pydantic turns annotated classes into validators: a request body is checked and converted before your handler runs, and replies are serialised from typed models.

HOW IT WORKS

- `ChatRequest.question` must be 3–1,000 characters; `DecisionRequest.decision` must be one of the three decisions.

- A bad body never reaches the handler; the client gets 422 listing every failing field.
- Replies carry only plain Python values. A pandas `Int64` or NumPy float must be converted first, or serialisation fails.

EXAMPLE

```
class DecisionRequest(BaseModel):
    decision: Literal["ORDER REPAIR", "INSPECT ON SITE", "REFER FOR COORDINATION"]
    decision_reason: str = Field(min_length=1)
```

Where it is used in StreetCare AI: `api/schemas.py`: SignInRequest, QueueRow, ReportDetailResponse, ReviewRequest, DecisionRequest, ReviewRecordResponse, ReviewCallsResponse, ChatRequest/Response, ChatTurnResponse.

INTERVIEW QUESTIONS

Q. Why convert NumPy values before replying?

A. JSON encoders don't know `numpy.int64` or `pd.NA`. `describe_record` turns them into plain ints, floats, strings and None.

19 Password hashing with PBKDF2 & salts

Security

Definition. Storing a **one-way, salted, deliberately slow** hash of a password instead of the password, so a stolen database does not reveal it.

HOW IT WORKS

- A random 16-byte **salt** per account: identical passwords get different hashes.
- **PBKDF2-HMAC-SHA256** with 200,000 rounds makes each guess expensive.
- Sign-in recomputes the hash with the stored salt and compares in constant time (`hmac.compare_digest`).
- The same password and salt always give the same hash; a different salt gives a different one.

EXAMPLE

```
digest = hashlib.pbkdf2_hmac("sha256", plain_password.encode(), bytes.fromhex(password_salt), 200_000)
return digest.hex()
```

Where it is used in StreetCare AI: `access/password_security.hash_password`; config `PASSWORD_HASH_ALGORITHM = "sha256"`, `PASSWORD_HASH_ROUNDS = 200000`, `PASSWORD_SALT_BYTES = 16`.

INTERVIEW QUESTIONS

Q. Why not plain SHA-256?

A. It is built to be fast, so attackers can try billions of guesses a second. A key-derivation function with many rounds (PBKDF2, bcrypt, scrypt, Argon2) slows each guess down.

Q. Hashing vs encryption?

A. Encryption can be reversed with a key; hashing cannot. Passwords are hashed, never encrypted.

20 Session tokens

Security API

> **Definition.** A long random string issued at sign-in and sent with every request to prove who the caller is. The server looks it up to find the account.

HOW IT WORKS

- `secrets.token_hex(24)` : unguessable.
- Sent in the `X-Session-Token` header.
- Stored on the account: signing in elsewhere replaces it, ending the old session (`SESSION_ENDED_MESSAGE`).
- Signing out clears it.

> **Where it is used in StreetCare AI:** `update_session_token` in the repository; `authorize_request(request, session_token)` in `access/permissions.py` .

INTERVIEW QUESTIONS

Q. Session tokens vs JWT?

A. A session token is opaque and checked against the database, so it can be revoked instantly. A JWT is self-contained and stateless, but harder to revoke before it expires.

Q. Why one active session per account?

A. It makes a leaked token stop working as soon as the real user signs in again, and keeps the audit trail clear.

21 Role-based access control (RBAC)

Security API

> **Definition.** Granting permissions by **role**, then checking the permission each route needs before running it.

HOW IT WORKS

- `PERMISSION_BY_ROUTE` maps (method, path) → permission.
- `ROLE_PERMISSIONS` maps role → permissions.
- `ROLE_PAGE_ACCESS` maps role → dashboard pages (a convenience; the API enforces).
- A refusal says *"The {role} role does not include the {permission} permission"*.

EXAMPLE

Permission	Highways Inspector	Street Care Coordinator	Customer Services Adviser
queue	✔	✔	–

review	✔	✔	–
decision	✔	–	–
audit	✔	✔	–
chat	✔	✔	✔

Where it is used in StreetCare AI: `access/permissions.py`; 16 seed accounts in `SEED_USER_ACCOUNTS`.

INTERVIEW QUESTIONS

Q. Authentication vs authorisation?

A. Authentication: who are you (password, token). Authorisation: what may you do (role permissions, district).

22 Separation of duties & district authorisation

Security

Definition. Requiring that **no one person** both starts and approves an action, and that an approver is responsible for the area concerned.

HOW IT WORKS

- Only a **Highways Inspector** may decide.
- Only one **appointed for the report's district** (`INSPECTION_DISTRICT_STREETS` maps districts to streets).
- Never the person who **opened** the review (`OWN_REVIEW_MESSAGE`).
- Two inspectors per district, so one can always decide what the other opened.

Where it is used in StreetCare AI: `find_decision_refusal(review_decision, report, screening, opened_by)` in `tools/decision_tools.py`; `authorize_district`.

INTERVIEW QUESTIONS

Q. Why does this matter in a council?

A. It stops one person both raising and approving spending on a street, which is a basic control against error and fraud, and it is auditable in `review_decisions`.

23 Concurrency: locks & conflicts

API

Definition. When two requests may change the same thing at once, a **lock** lets only one through at a time, and a **conflict check** refuses the second if the state has already moved on.

HOW IT WORKS

- One `threading.Lock` per queued report (`report_locks`), not one global lock.
- Under the lock: `find_review_conflict` checks the review status and any decision the checkpoint already holds.

- Already decided → 409; a *different* decision in progress → 409; the *same* decision sent again after a failure finishes the run.
- All of this happens before resuming, so no LLM call is wasted.

> **Where it is used in StreetCare AI:** `submit_review_decision` in `api/routes.py`; `find_review_conflict` in `tools/decision_tools.py`.

INTERVIEW QUESTIONS

Q. Why per-report locks?

A. Two inspectors deciding different reports shouldn't wait for each other; only decisions on the same report must be serialised.

24 SQL, SQLite & the SQLAlchemy ORM

Data API

> **Definition.** SQLite is a full SQL database in one file. An **ORM** maps tables to Python classes and rows to objects, generating safe, parameterised SQL.

HOW IT WORKS

- Six tables: `user_accounts`, `agent_runs`, `report_reviews` (37 columns), `review_decisions`, `llm_calls`, `chat_turns`.
- Names are exact and lower case: the marking suite and the replies read them.
- Only a column marked nullable accepts nothing.
- All writes go through `persistence/repository.py`.

EXAMPLE

```
class LlmCall(Base):
    __tablename__ = "llm_calls"
    call_id = Column(Integer, primary_key=True)
    run_id = Column(String, ForeignKey("agent_runs.run_id"), nullable=False)
    agent_name = Column(String, nullable=False)
    total_tokens = Column(Integer, nullable=False)
    latency_milliseconds = Column(Integer, nullable=False)
```

> **Where it is used in StreetCare AI:** `persistence/database.py`, `models.py`, `repository.py` → `database/streetcare.db`.

INTERVIEW QUESTIONS

Q. ORM pros and cons?

A. Pros: less SQL, safer by default (no injection), portable. Cons: can hide inefficient queries and is harder to tune.

Q. What is SQL injection?

A. User text pasted into SQL that changes the query. Parameterised queries (which the ORM uses) prevent it.

25 Transactions & atomic writes

Data API

> **Definition.** A **transaction** groups several writes so they all happen or none do (commit or roll back), so the database is never half-updated.

HOW IT WORKS

- Node 6 writes the writer's call, the review (AWAITING_DECISION) and the run (AWAITING_DECISION, all calls) in **one** transaction.
- Node 10 writes the decision, the review's outcome and the closed run (COMPLETE, total calls) in one transaction.
- On any error: roll back, mark the run FAILED, and let the same decision be sent again.

EXAMPLE

```
with session_factory() as session, session.begin():   # commits on success, rolls back on error
    insert_llm_call(session, run_id, call_record)
    insert_report_review(session, review_record)
    update_agent_run(session, run_id, AGENT_RUN_STATUS_AWAITING_DECISION, llm_call_count)
```

> **Where it is used in StreetCare AI:** `prepare_report_recommendation_node` and `record_report_outcome_node`.

INTERVIEW QUESTIONS

Q. What could go wrong without a transaction here?

A. A review saved without its run update, or a decision saved while the review still says AWAITING_DECISION. The audit trail would contradict itself.

26 Thin clients: Streamlit over an API

API

> **Definition.** A user interface that only **displays** data and **sends** requests, holding no business rules of its own.

HOW IT WORKS

- Every call goes through `call_service(method, path, query, payload, waiting_message, failure_message)`.
- A 300-second timeout (`API_REQUEST_TIMEOUT_SECONDS`), because opening a review makes five LLM calls.
- Pages: sign in · report queue · report detail · decision queue · audit trail · street care chat.
- Each role sees only its pages; hiding is a convenience, security is the API's job.

Where it is used in StreetCare AI: `dashboard/*.py` and `run_dashboard.py` on port 8080.

INTERVIEW QUESTIONS

Q. Why not compute anything in the dashboard?

A. Then two implementations could disagree, and anyone calling the API directly would bypass them. One source of truth: the service.

PART 4

Machine learning

Three models that estimate, are scored honestly, and are refused if they are not good enough.

27 Supervised learning: classification & regression

ML

Definition. Learning a mapping from inputs (features) to a known answer (label) from past examples. **Classification** predicts a category; **regression** predicts a number.

EXAMPLE

Model	Learns from	Label	Kind
Repeat	report history	`reported_again` (Yes = 1, No = 0)	binary classification → probability
Crew hours	repair logs	`actual_crew_hours`	regression → hours
Sentence	labelled sentences	`sentence_type` (9 classes)	multi-class text classification

Where it is used in StreetCare AI: `modeling/repeat_model.py`, `crew_hours_model.py`, `sentence_model.py`.

INTERVIEW QUESTIONS

Q. Why predict a probability rather than yes/no for repeats?

A. A probability can be banded (LOW / MEDIUM / HIGH) and explained, and the inspector weighs it; a bare yes/no hides how sure the model is.

28 Held-out evaluation & train/eval files

ML

Definition. Scoring a model on data it **never learned from**, to estimate how it will do on new reports. The gap between the training score and the held-out score shows how much it memorised.

HOW IT WORKS

- History and repair logs come as separate `_train` and `_eval` files: fit on train only, score on eval only.
- Sentences come as one file: 20% is held out (`SENTENCE_HELD_OUT_SHARE`), seeded with 1313.

- Report **held-out score, training score and the gap** (printed with its sign) for each model.

EXAMPLE

```
repeat          held-out ROC AUC N.NNNN  (train N.NNNN, gap +N.NNNN)
crew hours      held-out R2 N.NNNN  MAE N.NNNN hours  (train N.NNNN, gap +N.NNNN)
sentence        held-out macro F1 N.NNNN  (train N.NNNN, gap +N.NNNN)
```

Where it is used in StreetCare AI: `model_evaluation.py` : `evaluate_classifier` , `evaluate_regressor` , `evaluate_text_classifier` ("No function here fits anything.")

INTERVIEW QUESTIONS

Q. Why fix the random seed?

A. So a score change comes from a code change, not from luck of a split. Every random choice uses `RANDOM_SEED = 1313` .

29 Data leakage

ML

Definition. When information that would not be available at prediction time (or that comes from the evaluation data) sneaks into training, so scores look great and then collapse in use.

HOW IT WORKS

- **Train-eval contamination**: fitting the imputer, scaler or TF-IDF vocabulary on all data before splitting.
- **Target leakage**: a column only known after the outcome, e.g. a repeat cause on a defect that came back.
- Fix: preparation lives **inside** the pipeline, fitted on training rows only; post-outcome columns are not features.

Where it is used in StreetCare AI: `REPEAT_NUMERIC_FEATURES` / `REPEAT_CATEGORICAL_FEATURES` list only what a report knows when it arrives, plus counts and flags the screening derives from it.

INTERVIEW QUESTIONS

Q. How would you spot leakage?

A. A suspiciously high score, one feature with huge importance, or a big drop between held-out and live. Ask: would I know this value at the moment of deciding?

30 Preprocessing: imputation, scaling, one-hot

ML

Definition. Turning raw columns into the complete, numeric, comparable form models need.

HOW IT WORKS

- **Imputation**: numbers with the training **median** (`NUMBER_FILL_STRATEGY`), categories with the **most frequent** value.

- **Scaling**: z = (x − mean) / std so depths in mm and counts of 0–5 sit on one scale (matters for logistic regression).
- **One-hot**: `defect_type = Footway` → `is_Footway = 1` ; `handle_unknown="ignore"` stops a new category crashing live scoring.

EXAMPLE

defect_type	Carriageway	Footway	Drainage	Vegetation	Street lighting
Footway	0	1	0	0	0
Drainage	0	0	1	0	0

Where it is used in StreetCare AI: `build_model_pipeline` in `feature_builder.py`. Real gaps (water depth on a footway report) are filled here, not in the clean CSV.

INTERVIEW QUESTIONS

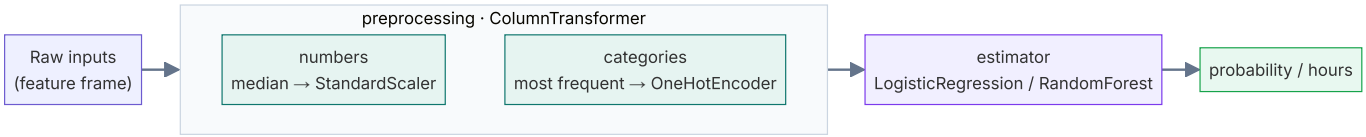
Q. Why not label-encode categories as 1–5?

A. It invents an order (Street lighting > Footway) that a linear model would treat as meaningful.

31 scikit-learn Pipeline & ColumnTransformer

ML

Definition. A **Pipeline** chains preparation and a model into one object with one `fit` / `predict` ; a **ColumnTransformer** applies different preparation to different columns.



One object: prepare + estimate. Saved to disk as a whole

EXAMPLE

```
Pipeline([(config.PREPROCESSING_STEP_NAME, ColumnTransformer([...])),   # "preprocessing"
          (config.MODEL_STEP_NAME, estimator)])                          # "estimator"
```

Where it is used in StreetCare AI: Both tabular models; the sentence model is a Pipeline of TF-IDF ("preprocessing") → LogisticRegression ("estimator").

INTERVIEW QUESTIONS

Q. Why pipelines?

A. No leakage, one object to save and load, identical preparation at training and scoring time.

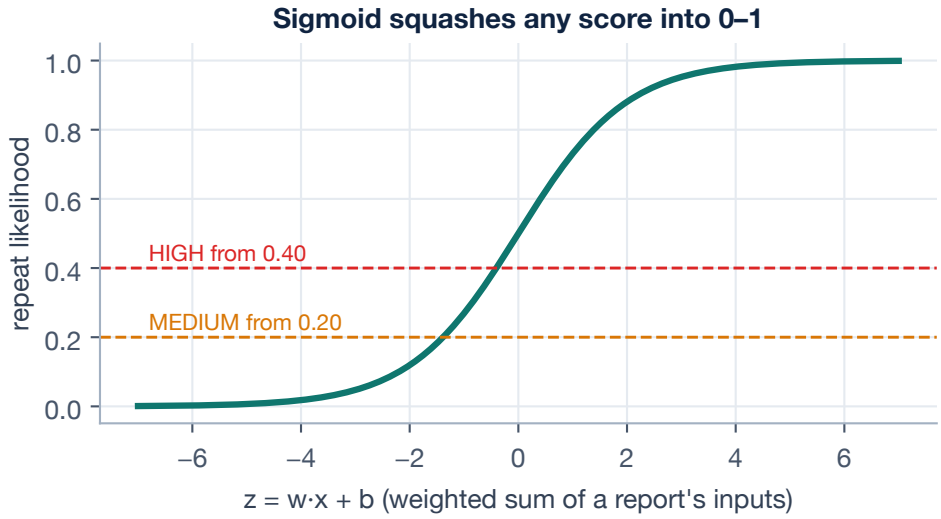
32 Logistic regression

ML

Definition. A linear model for classification: a weighted sum of the inputs passed through the **sigmoid** to give a probability between 0 and 1.

HOW IT WORKS

- $z = w_1x_1 + w_2x_2 + \ldots + b$; $P(\text{repeat}) = 1 / (1 + e^{-z})$.
- Weights are learned by minimising log loss; `max_iter = 3000` gives the solver room to converge.
- Interpretable: a positive weight on `requirements_not_met_count` raises the odds of a repeat.
- Regularisation (L2 by default) discourages over-large weights.



The sigmoid turns any score into a likelihood; the dashed lines are StreetCare's band thresholds

Where it is used in StreetCare AI: The repeat model (`LOGISTIC_REGRESSION_MAX_ITERATIONS = 3000`) and the sentence model's estimator.

INTERVIEW QUESTIONS

Q. Why is it called regression if it classifies?

A. It regresses the log-odds linearly; turning the probability into a class (or band) is a separate step.

33 Random forest regression

ML

Definition. An **ensemble** of decision trees, each trained on a bootstrap sample of rows and a random subset of features at each split; the forest's prediction is the average of the trees.

Analogy: Asking 300 experienced crew supervisors, each of whom has seen a different set of past jobs, and averaging their estimates. Individual mistakes cancel out.

HOW IT WORKS

- `CREW_HOURS_TREE_COUNT = 300` trees.

- `CREW_HOURS_MIN_SAMPLES_LEAF = 10`: every leaf averages at least 10 repairs, so it can't memorise one odd job.
- `CREW_HOURS_MAX_FEATURES = 0.7`: each split considers 70% of the features, decorrelating the trees.
- Trees don't need scaling, but the shared pipeline scales anyway, harmlessly.

> **Where it is used in StreetCare AI:** `train_crew_hours_model` → `output/models/crew_hours_model.pkl`; must reach $R^2 \geq 0.86$ on held-out logs.

INTERVIEW QUESTIONS

Q. Bagging vs boosting?

A. Bagging (random forest) trains trees independently and averages them, reducing variance. Boosting trains trees in sequence, each fixing the last one's errors, reducing bias.

Q. Why can a forest's forecast never be below the fewest hours logged?

A. A tree's leaf predicts an average of training targets, so a forest can't predict below the smallest training value; the specification checks this.

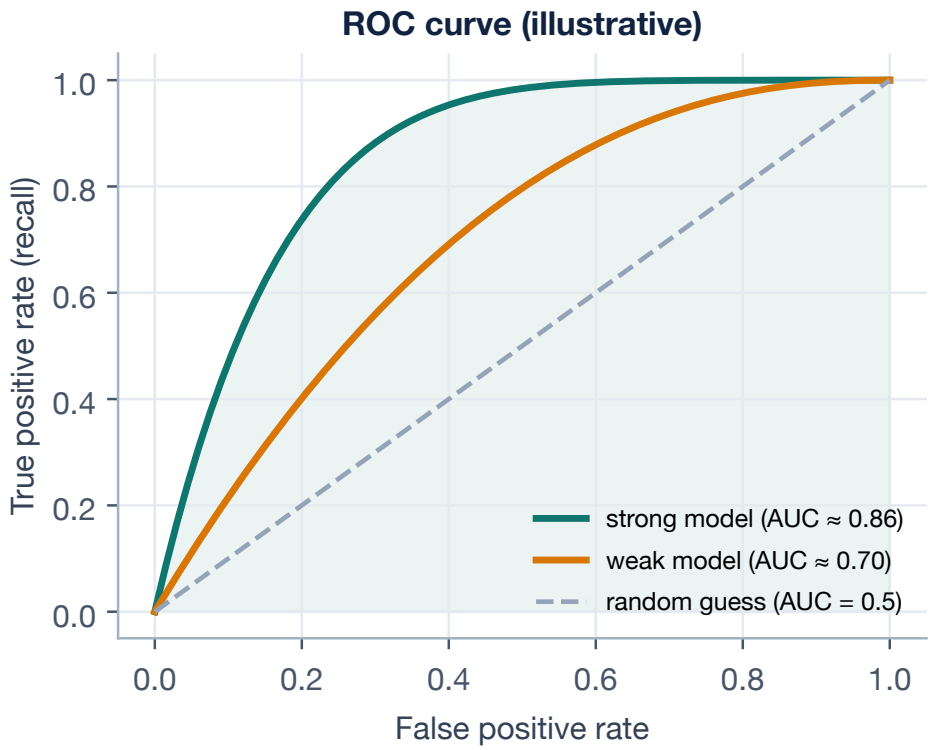
34 ROC curve & AUC

ML

> **Definition.** The ROC curve plots true-positive rate against false-positive rate for every threshold. **AUC** is the area under it: 1.0 = perfect ranking, 0.5 = random.

HOW IT WORKS

- Threshold-free: it measures how well the model **ranks** repeats above non-repeats.
- Robust to class imbalance, unlike accuracy.
- AUC 0.80 means: pick one defect that came back and one that didn't; 80% of the time the model gives the returning one the higher score.



Illustrative curves; StreetCare's repeat model must reach a held-out AUC of at least 0.80

Where it is used in StreetCare AI: `evaluate_classifier`; `MINIMUM_REPEAT_ROC_AUC = 0.80`.

INTERVIEW QUESTIONS

Q. Why not accuracy?

A. If most defects never return, a model that always says "no repeat" is very accurate and useless. AUC measures ranking.

35 R^2 and MAE for regression

ML

Definition. **R^2** is the share of the variation in actual hours the model explains (1 = perfect, 0 = no better than always predicting the mean). **MAE** is the average absolute miss, in hours.

HOW IT WORKS

- $R^2 = 1 - \Sigma(\text{actual} - \text{predicted})^2 / \Sigma(\text{actual} - \text{mean})^2$.
- MAE = mean(|actual − predicted|), easy to explain: "off by 0.4 hours on average".
- The crew hours model must reach $R^2 \geq 0.86$ and its MAE must be below 0.75 × a simple baseline's.

Where it is used in StreetCare AI: `evaluate_regressor`; `MINIMUM_CREW_HOURS_R2 = 0.86`.

INTERVIEW QUESTIONS

Q. Why report both?

A. R^2 is scale-free and good for comparing models; MAE is in hours, which is what a supervisor plans with.

36 Macro F1 for multi-class text

ML NLP

Definition. F1 is the harmonic mean of precision and recall. **Macro F1** averages F1 equally over every class, so rare sentence types count as much as common ones.

HOW IT WORKS

- Precision: of sentences labelled Measurement, how many really are.
- Recall: of real Measurement sentences, how many were found.
- Macro averaging stops the model from scoring well by ignoring the rare *Incident* or *Previous reports* classes.

Where it is used in StreetCare AI: `evaluate_text_classifier`; `MINIMUM_SENTENCE_MACRO_F1 = 0.95`; every type must be predicted for some held-out sentence.

INTERVIEW QUESTIONS

Q. Macro vs weighted F1?

A. Weighted F1 averages by class size and can hide a rare class doing badly. Macro treats all nine equally.

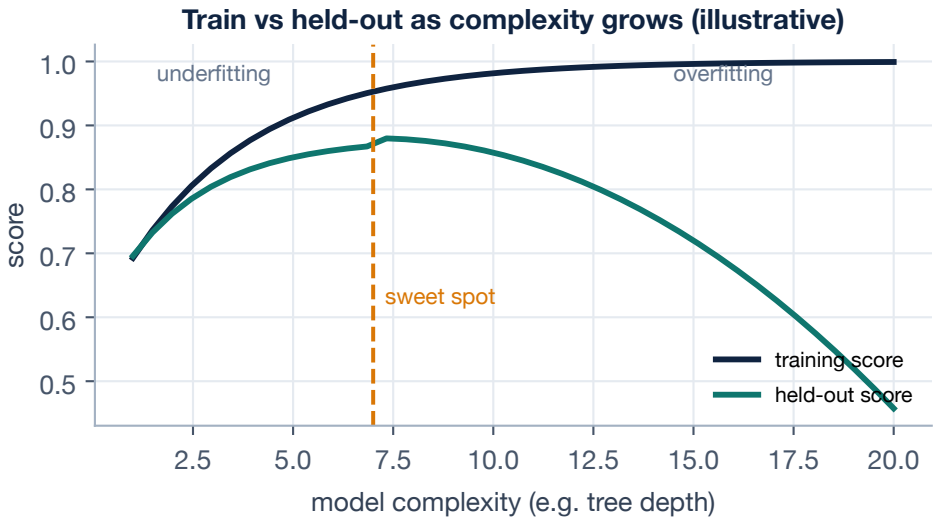
37 Overfitting & the train vs held-out gap

ML

Definition. Overfitting: the model memorises training noise and does worse on new data. **Underfitting**: too simple to capture the pattern.

HOW IT WORKS

- Sign of overfitting: training score ≫ held-out score.
- Cures in StreetCare: `min_samples_leaf = 10`, `max_features = 0.7`, regularised logistic regression, more data.
- The repeat model's mean held-out score must be within 0.03 of its training score.



As complexity grows, the gap between training and held-out scores widens

> **Where it is used in StreetCare AI:** `model_metrics.json` stores held-out, train and gap for every model.

INTERVIEW QUESTIONS

Q. High bias or high variance, how to tell?

A. Both scores low → high bias (underfit). Training high, held-out much lower → high variance (overfit).

38 Behavioural model tests (monotonic checks)

ML Testing

> **Definition.** Checking that a model **reacts in the expected direction** when one input changes, not only that it scores well on average.

HOW IT WORKS

- Repeat: change one requirement from Met to Not met → score rises for ≥ 95% of held-out reports.
- Repeat: add one conflicting live permit → score rises for ≥ 95%.
- Crew hours: traffic-sensitive street instead of local road → forecast lengthens for ≥ 90% of repairs on local roads.
- Crew hours: one more conflicting permit → forecast lengthens for ≥ 90%.
- Across the live queue: likelihoods fall in all three bands; overruns are expected for some reports and not others.

EXAMPLE

```
changed = features.copy()
changed["requirements_not_met_count"] += 1
changed["requirements_met_count"] -= 1
rise = (model.predict_proba(changed)[:, 1] > model.predict_proba(features)[:, 1]).mean()
assert rise ≥ 0.95
```

Where it is used in StreetCare AI: The requirements of `train_repeat_model` and `train_crew_hours_model`.

INTERVIEW QUESTIONS

Q. Why aren't good metrics enough?

A. A model can score well by leaning on a quirk of the data and still move the wrong way for the inputs the business cares about. Behavioural checks catch that.

39 Model gating by minimum score

ML

Definition. Refusing to save or serve a model unless it reaches a **minimum** on held-out data.

HOW IT WORKS

- Train all three, evaluate all three, check every minimum, **then** save.
- Any model below its minimum stops the run with a refusal naming it; nothing is written to `output/models/`.
- The service refuses to start without the saved models, so a bad model can never be served.

EXAMPLE

```
every model reaches the minimum the configuration sets for it
metrics written to output/model_metrics.json
```

Where it is used in StreetCare AI: `run_model_training.py`; `MINIMUM_REPEAT_ROC_AUC = 0.80`, `MINIMUM_CREW_HOURS_R2 = 0.86`, `MINIMUM_SENTENCE_MACRO_F1 = 0.95`.

INTERVIEW QUESTIONS

Q. Why check all minimums before saving any model?

A. So the saved set is always consistent: never a new repeat model beside an old crew hours model.

40 Model persistence & feature schemas

ML

Definition. Saving a fitted pipeline to disk so the service can score without retraining, together with the exact list of input columns, in order.

HOW IT WORKS

- `joblib.dump(model, path, compress=3)`: the whole pipeline, preparation included.
- `repeat_features.json` / `crew_hours_features.json`: column names in fitted order (JSON, indent 2).
- At start-up the service loads the models and schemas once; each live report is laid out in schema order.

EXAMPLE

```
model = load_model(config.REPEAT_MODEL_FILE)
names = load_feature_schema(config.REPEAT_SCHEMA_FILE)
row = build_feature_frame(profile_frame, numeric, categorical)[names]
probability = model.predict_proba(row)[0, 1]
```

Where it is used in StreetCare AI: `modeling/model_registry.py` ("Nothing else in the project writes to output/models/").

INTERVIEW QUESTIONS

Q. Risks of pickle / joblib?

A. Loading runs code, so only load trusted files; pickles can break across library versions, so pin versions or retrain.

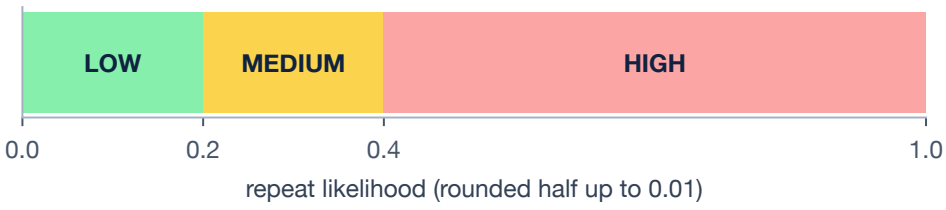
41 From probability to band, and estimates that inform

ML

Definition. Turning a model output into a **human-readable figure** with fixed rounding and fixed wording, and keeping it advisory.

HOW IT WORKS

- Likelihood rounded half up to 0.01; band HIGH from 0.40, MEDIUM from 0.20, LOW below.
- Crew hours rounded to 0.1; an overrun if expected > 1.3 × planned; no plan → its own sentence.
- Neither figure orders the queue, sets the assessment, appears in outgoing documents or reaches customer services.



Repeat likelihood bands (config HIGH_LIKELIHOOD_MINIMUM and MEDIUM_LIKELIHOOD_MINIMUM)

Where it is used in StreetCare AI: `estimate_repeat_likelihood`, `estimate_expected_crew_hours`, `describe_expected_crew_hours` in `tools/prediction_tools.py`.

INTERVIEW QUESTIONS

Q. Why fixed templates for the wording?

A. So every review says it the same way, the dashboard can show it verbatim, and an agent can't reword a figure into a promise.

PART 5

Natural language processing (NLP)

Reading every sentence a resident, caller or inspector wrote about a defect.

42 Text cleaning: markup, entities, whitespace, capitals

NLP

Definition. Turning raw free text into the plain words a person reads, so the model learns from meaning and not wrapping.

HOW IT WORKS

- Remove tags matched by `MARKUP_PATTERN` (e.g. `<span>`, `<br/>`).
- Decode character entities (`&` → &).
- Collapse every run of whitespace, including no-break spaces, to one space; trim.
- A sentence with letters but no lower-case letter is lower-cased; length is recounted.

EXAMPLE

```
raw   : "<span>I am reporting this for my father. Rain water runs <br/>down the bank</span>"
clean : "I am reporting this for my father. Rain water runs down the bank"
```

Where it is used in StreetCare AI: `convert_markup_to_plain_text`, `lower_text_written_in_capitals`; used for training sentences and live documents.

INTERVIEW QUESTIONS

Q. Why lower-case only all-caps sentences?

A. Case carries meaning elsewhere (street names, references). An all-caps sentence is the only case where restoring it is safe.

43 Sentence splitting

NLP

Definition. Cutting a document into sentences so each can be labelled and checked on its own.

HOW IT WORKS

- Statements split where `STATEMENT_SENTENCE_PATTERN = (?⩽[.!?])\s+` matches.
- Policy text uses `SENTENCE_BOUNDARY_PATTERN`, which avoids splitting after a numbered heading like "4.".
- A document with no text is refused (`NO_STATEMENT_TEXT_MESSAGE`).

Where it is used in StreetCare AI: `read_document_evidence(sentence_model, documents)` in `tools/evidence_tools.py`.

INTERVIEW QUESTIONS

Q. What breaks naive sentence splitting?

A. Abbreviations and decimals ("2.1 m"). The look-behind requires punctuation followed by whitespace, so "2.1" stays whole.

44 Tokenisation, n-grams & TF-IDF

NLP

Definition. Tokenisation splits text into words; n-grams are runs of n words; **TF-IDF** weights a word high when it is frequent in this sentence but rare across all sentences.

HOW IT WORKS

- tf-idf(t, d) = tf(t, d) × log(N / df(t)).
- "the" appears everywhere → weight ≈ 0; "deep" or "mm" → telling.
- Each sentence becomes a long sparse vector, ideal input for a linear classifier.
- Fitted on training sentences only, inside the pipeline.

EXAMPLE

Word	In this sentence	Across sentences	Weight
"the"	often	almost all	≈ 0
"mm"	once	Measurement sentences	high
"hurt"	once	Incident sentences	high

Where it is used in StreetCare AI: The sentence model's `preprocessing` step.

INTERVIEW QUESTIONS

Q. TF-IDF vs embeddings?

A. TF-IDF is sparse, fast and explainable but knows nothing of meaning ("pavement" ≠ "footway"). Embeddings capture similarity of meaning; StreetCare uses them for search.

45 Text classification: the nine sentence types

NLP ML

Definition. Predicting one label for each piece of text: here, which of nine types each sentence of a report's documents is.

EXAMPLE

Sentence type	Bears on requirement kinds	Example
Measurement	depth, width, water depth/reach, clear height, column lean	"It is 45 mm deep."
Location	location fixed, column number, highway land	"Outside no. 69."

Photograph	photograph	"The photo shows a coin by the hole."
Weather and water	water depth, water reach	"Rain water collects here."
Hazard · Incident · Traffic and use · Previous reports · Background	none	"I fell over here."

Where it is used in StreetCare AI: `train_sentence_model` → `sentence_model.pkl`; `SENTENCE_TYPE_REQUIREMENT_KINDS`; `link_evidence_to_requirements`.

INTERVIEW QUESTIONS

Q. Why type sentences before extracting figures?

A. Only Measurement sentences are searched for figures, so a phone number or house number in a Location sentence is never mistaken for a depth.

46 Rule-based information extraction

NLP

Definition. Pulling structured facts out of free text with patterns, when the format is well defined.

HOW IT WORKS

- Four patterns: `N mm deep`, `N mm wide`, `N mm of standing water`, `N m clear` (case-insensitive).
- Each figure is captured as text and turned into an exact Decimal.
- Figures in cm, inches or feet are **not** held against the report: the guide says the reviewer notes them.

EXAMPLE

```
"There is only 2.1 m clear under the branches."
→ {"clear_height_m": Decimal("2.1")}
```

Where it is used in StreetCare AI: `STATEMENT_MEASUREMENT_PATTERNS`; `find_disagreements(findings, report)`.

INTERVIEW QUESTIONS

Q. Rule-based or learned extraction?

A. Rules are precise and explainable for fixed formats like these; learned NER handles open-ended entities but needs labelled data.

47 Comparing evidence with the record: disagreement flags

NLP Data

Definition. Checking each figure a document gives against the report's own reading of the same answer, and **flagging** (never changing) a mismatch.

HOW IT WORKS

- Report left the answer blank → `DESCRIBES BLANK ANSWER`.

- Otherwise a figure that differs, compared unrounded and exactly → `DISAGREES WITH FORM`.
- The report's answer still decides the requirement; the flag tells the inspector what to settle on site.

EXAMPLE

```
report: clear_height_m = 2.4
statement: "only 2.1 m clear"  →  DISAGREES WITH FORM
```

Where it is used in StreetCare AI: `find_disagreements`; shown on the report detail page and in the inspection request.

INTERVIEW QUESTIONS

Q. Why not update the report with the statement's figure?

A. A resident's sentence is evidence, not a measurement on the form. Changing answers would let unverified text decide a repair.

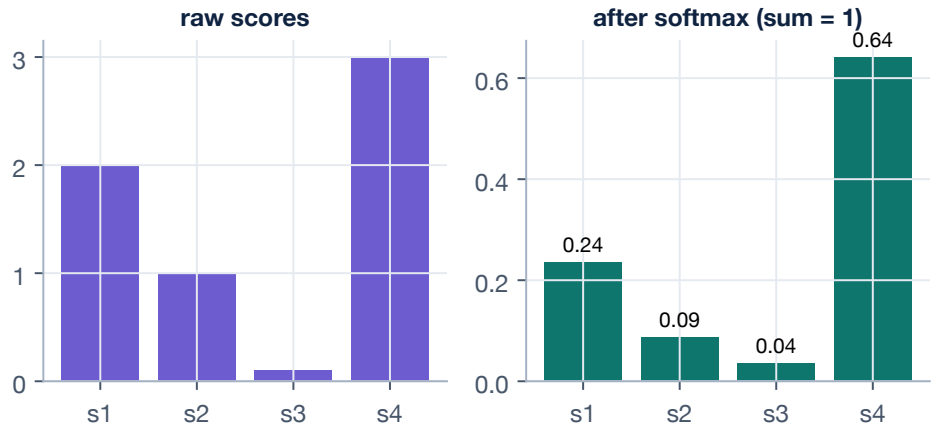
48 Attention & transformers, in brief

NLP

Definition. Attention lets each token take a weighted average of all others, weighted by relevance: softmax($QK^T/\sqrt{d}$)·V. **Transformers** stack attention and feed-forward layers; they power embedding models and LLMs.

HOW IT WORKS

- $Q \cdot K^T$ scores how much each token should look at each other token.
- $\div\sqrt{d}$ keeps scores from growing with dimension so softmax doesn't saturate.
- Softmax turns scores into weights that sum to 1.
- Encoder-only (MiniLM) → embeddings; decoder-only (Gemini) → generation.



Softmax turns any scores into positive weights that add up to 1

Where it is used in StreetCare AI: Not trained here, but used twice: `all-MiniLM-L6-v2` (6 layers, 384-dimensional vectors) embeds the PDFs; Gemini writes the review prose.

INTERVIEW QUESTIONS

Q. Why did transformers replace RNNs?

A. Attention links any two positions directly and processes the sequence in parallel, so it captures long context and trains much faster.

PART 6

Large language models & GenAI safety

Letting Gemini explain and write, while code keeps every fact and figure.

49 Large language models (LLMs)

GenAI

Definition. Very large transformer models trained to predict the next token, which lets them follow instructions, explain and write in natural language.

HOW IT WORKS

- Tokens: text is split into sub-word pieces; cost and limits are counted in tokens.
- Temperature: randomness. StreetCare uses 0.2 for steady, repeatable wording.
- `GEMINI_MAX_OUTPUT_TOKENS = 8192`.
- Accessed with an API key through `google-genai`; key and model name are read **at call time** from `.env`.

Where it is used in StreetCare AI: `agentic/language_model.call_language_model(prompt_text, agent_name)`; missing key or model → refused, naming both settings.

INTERVIEW QUESTIONS

Q. What is hallucination?

A. Confident output not supported by the input. StreetCare limits it by giving facts in, asking for prose out, validating fields, and keeping every figure in code.

50 Prompt engineering & labelled-field output

GenAI

Definition. Designing what you send to an LLM so its answer is correct, constrained and machine-readable.

HOW IT WORKS

- Speak for the Street Care Office of Wrenford Borough Council; give the role, the task and the facts.
- Tell the agent to use only what it is given.
- Ask for a fixed layout of **labelled fields**, e.g. `RECOMMENDATION:` and `RECOMMENDED DECISION:`.
- The recommended decision must be one still open for the report, in upper case.

EXAMPLE

```
PRIORITY: ...
CHECKS: ...
RATIONALE: ...
```

Where it is used in StreetCare AI: `agentic/agents.py` : seven agents, each with its own fields (e.g. evidence analyst: EVIDENCE FOUND · DISAGREEMENTS · CHECKS TO MAKE · EVIDENCE SUMMARY).

INTERVIEW QUESTIONS

Q. Why labelled fields instead of free text?

A. Code needs fields it can rely on to store and display. Free text hides missing parts and breaks parsers.

51 Validating LLM output & refusing bad replies

GenAI

Definition. Treating an LLM reply as **untrusted input**: parse it strictly and refuse it if it doesn't fit, never patching it up.

HOW IT WORKS

- Refuse an empty reply, one that did not finish normally, or one that doesn't report token counts.
- `parse_labelled_response` : read each field ignoring case and edge decoration (`*#`_~`); refuse missing, duplicate or extra fields.
- Refuse a single-line field (subject, headline, decision) that runs over several lines.
- Only an **accepted** reply is recorded in `llm_calls` .

Where it is used in StreetCare AI: `agentic/language_model.py` ; refusals name the agent.

INTERVIEW QUESTIONS

Q. Why not repair a slightly broken reply?

A. A repaired reply is code guessing what the model meant, the opposite of the project's "refuse, never guess" rule.

52 Token & latency accounting

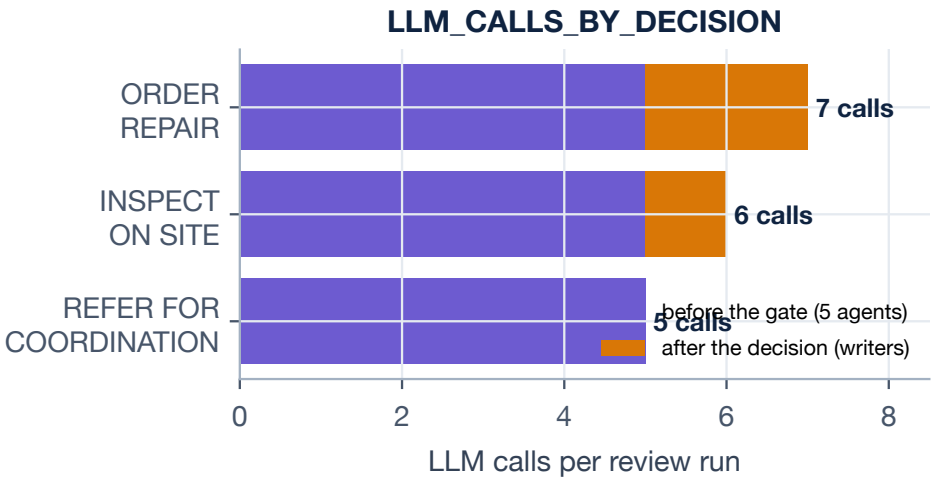
GenAI API

Definition. Recording what every LLM call cost (tokens) and how long it took (milliseconds), against the run it belongs to.

HOW IT WORKS

- Each call record: agent_name, model_name, prompt_tokens, response_tokens, total_tokens, latency_milliseconds.

- `GET /reviews/{id}/calls` shows call_count vs expected_call_count (5, 6 or 7), total tokens and total latency.
- A mismatch between count and expected is a red flag in the audit trail.



LLM calls per review run, by decision

Where it is used in StreetCare AI: `llm_calls` table; `LLM_CALLS_BEFORE_DECISION = 5`, `LLM_CALLS_BY_DECISION`.

INTERVIEW QUESTIONS

Q. Why measure latency?

A. A review makes five sequential calls; knowing which agent is slow tells you where to shorten a prompt, and justifies the dashboard's 300-second timeout.

53 Grounding: facts in, prose out

GenAI

Definition. Giving an LLM only facts that code has already established, and asking it to explain them, so every figure in its answer is traceable.

IN SIMPLE WORDS

The requirements analyst is told "Depth: Met (45 mm, at least 40)"; it never works the 40 out or reads the PDF itself.

HOW IT WORKS

- Tools establish: screening, permits, triggers, findings, flags, likelihood, crew hours.
- Knowledge sections are read by number (Policy §6, §7, §8; standards §6, §7, §8; guides §3) and passed in.
- The agent interprets; it calculates nothing of its own.

Where it is used in StreetCare AI: Every agent in `agentic/agents.py`; `gather_report_context` builds the facts.

INTERVIEW QUESTIONS

Q. Why not let the LLM read the standard and judge the report?

A. It could misread a limit or confuse two near-identical standards. Code reads limits exactly; the LLM only explains.

54 Output guardrails for documents sent out

GenAI Security

> **Definition.** Checks on what a model writes **before** it reaches a resident or a crew.

HOW IT WORKS

- Every document is signed `"Street Care Office, Wrenford Borough Council"`.
- No model figure: the words likelihood, probability and expected crew hours are forbidden.
- No unfilled template slots (`[` or `]`).
- The resident update names no member of staff.
- An order's two texts each hold at least 8 words and are not the same text.

> **Where it is used in StreetCare AI:** `DOCUMENT_SIGN_OFF`, `DOCUMENT_FORBIDDEN_TERMS`, `TEMPLATE_SLOT_CHARACTERS`, `NAME_IN_TEXT_PATTERN` in config; `draft_order_document`, `draft_resident_update`.

INTERVIEW QUESTIONS

Q. Why ban model figures from a works order?

A. A crew must not plan from an estimate presented as fact, and a resident must not read a probability about their street.

55 Privacy rules & role-aware answers

Security GenAI

> **Definition.** Rules that limit what each audience hears, enforced by code, not left to the model.

HOW IT WORKS

- Customer Services Adviser: a report's types, street and status; once ordered, where on the street and what the resident has been told.
- Asked if a defect will come back or how long it will take, an adviser hears `OUTCOME_NOT_PREDICTED_LINE` and nothing more.
- Inspector / Coordinator: the full review record, figures included.
- A street figure only from ≥ 5 reports (`GROUP_SIZE_MINIMUM`), counted from history by defect type; otherwise `GROUP_SIZE_LINE`.

> **Where it is used in StreetCare AI:** `chat/record_answer.py` : `answer_report_record`, `answer_street_figure`.

INTERVIEW QUESTIONS

Q. Why the minimum of five?

A. A figure built from one or two reports could identify the household that made them. It is a simple k-anonymity threshold.

PART 7

Retrieval & the knowledge base (RAG)

Making 16 council PDFs the single, traceable source of every rule.

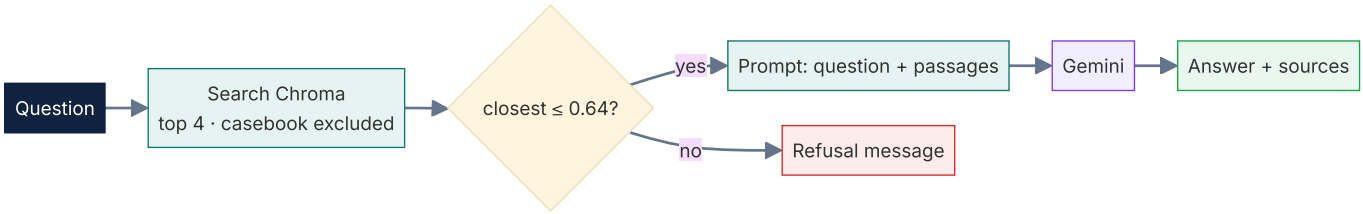
56 RAG: the big idea

RAG GenAI

Definition. Retrieval-Augmented Generation: find relevant passages in your own documents at question time and have the LLM answer from them, citing the sources.

HOW IT WORKS

- Knowledge stays current: change a PDF, re-index.
- Answers are verifiable: each names its source documents.
- If nothing relevant is close enough, refuse instead of guessing.



Retrieve, check relevance, then generate

Where it is used in StreetCare AI: `knowledge/` (indexing) and `chat/street_care_assistant.answer_from_documents` (answering).

INTERVIEW QUESTIONS

Q. RAG vs fine-tuning?

A. RAG gives facts at question time: best for rules that change, like council standards. Fine-tuning changes model weights: best for style or narrow skills.

57 PDF extraction & section splitting

RAG

Definition. Reading PDFs into text and cutting them along their own structure: here, numbered sections.

HOW IT WORKS

- Every document opens with WRENFORD BOROUGH COUNCIL, its title and one header line matched by `DOCUMENT_HEADER_PATTERN`.
- Sections are numbered from 1 and found by `SECTION_HEADING_PATTERN = (?m)^([0-9]{1,2})\. [A-Z]`.
- Expected: **16 documents, 136 sections**.

> **Where it is used in StreetCare AI:** `extract_text_from_directory`, `split_documents_into_sections`.

INTERVIEW QUESTIONS

Q. Why split by section rather than fixed size?

A. A section is a natural unit of meaning ("4. Requirements"), and the system reads sections by number.

58 Chunking with overlap

RAG

> **Definition.** Cutting long sections into smaller passages for embedding, with some characters repeated between neighbours so a sentence on a boundary isn't lost.

HOW IT WORKS

- `CHUNK_SIZE_CHARACTERS = 700`, `CHUNK_OVERLAP_CHARACTERS = 120`.
- Every passage keeps document name, section number and title as metadata.
- Small chunks: precise but may lose context. Large chunks: dilute the embedding and cost prompt tokens.

> **Where it is used in StreetCare AI:** `build_chunk_records(section_entries)`; the passage count is printed by indexing.

INTERVIEW QUESTIONS

Q. What does overlap fix?

A. A requirement sentence split across two chunks would be half in each; 120 characters of overlap keeps it whole in at least one.

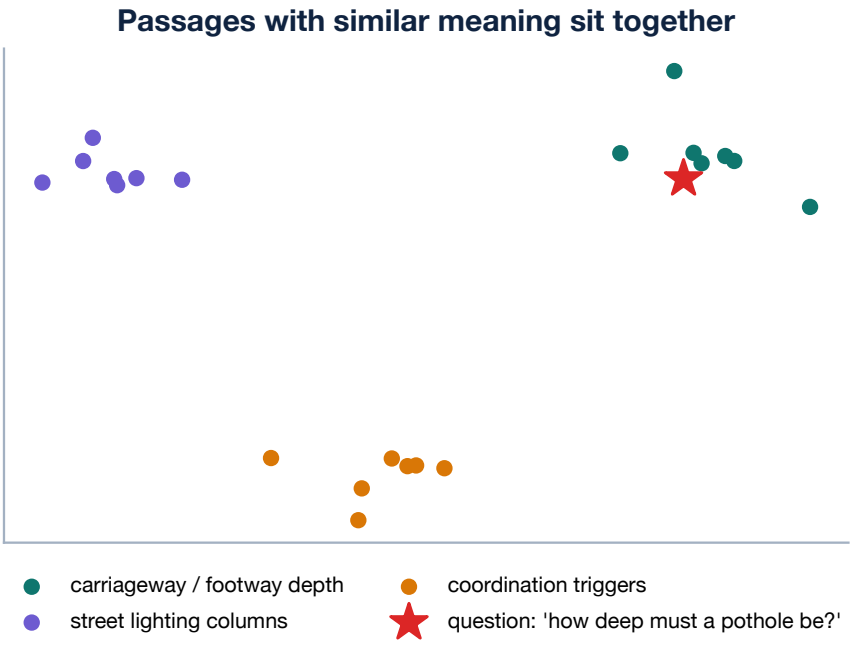
59 Sentence embeddings

RAG NLP

> **Definition.** Dense vectors that represent the **meaning** of a text, so texts with similar meaning sit close together even with different words.

HOW IT WORKS

- `all-MiniLM-L6-v2` (sentence-transformers), 384 dimensions, run on CPU.
- The **same** model must embed passages and questions.
- Downloaded once (~90 MB) on first indexing.



Illustrative 2-D view; the real vectors have 384 dimensions

Where it is used in StreetCare AI: `EMBEDDING_MODEL_NAME`, `EMBEDDING_DEVICE = "cpu"`; `build_vector_index`.

INTERVIEW QUESTIONS

Q. Word2Vec vs sentence-transformers?

A. Word2Vec gives one fixed vector per word. Sentence transformers produce context-aware vectors for whole passages, trained so similar sentences land close together.

60 Vector databases & cosine similarity (Chroma)

RAG

Definition. A database that stores embeddings with their text and metadata and finds the nearest vectors to a query quickly.

HOW IT WORKS

- Cosine similarity = cos(angle) between vectors; cosine **distance** = 1 − similarity (0 = identical).
- The collection is pinned to cosine (`VECTOR_DISTANCE_METRIC = "cosine"`).
- HNSW index: approximate nearest neighbours, fast.
- Persisted in `database/chroma_store`, collection `street_care_documents`.

Where it is used in StreetCare AI: `knowledge/vector_index.py` : `build_vector_index`, `open_vector_index`, `search_documents`.

INTERVIEW QUESTIONS

Q. Why cosine for text?

A. Direction carries meaning in embeddings; length mostly reflects text length. Cosine ignores length.

61 Exact section lookup vs similarity search

RAG

> **Definition.** Two ways to get text from the index: a **metadata lookup** (this document, this section number) and a **similarity search** (whatever is closest in meaning).

HOW IT WORKS

- The workflow always uses lookups: `read_document_section(collection, document_name, section_number)`, and checks the title matches.
- Only the chat uses similarity search.
- Why: the five standards share one template; similarity could return another type's identical-looking section.

EXAMPLE

```
collection.get(where={"$and": [{"document_name": "Defect-Standard-Footway.pdf"},
                               {"section_number": 6}]})
```

> **Where it is used in StreetCare AI:** `retrieval_tools`: `read_reviewer_checks`, `read_inspection_records`, `read_works_order_guidance`, `read_resident_update_guidance`, `read_policy_section`.

INTERVIEW QUESTIONS

Q. How does indexing prove lookups work?

A. It prints §6 of each standard; each must be its own standard's text, whole and without its heading. Two types showing the same text is a failure.

62 Relevance thresholds, sources & refusals

RAG GenAI

> **Definition.** Answering only when retrieval found something genuinely close, listing the documents used, and refusing otherwise.

HOW IT WORKS

- Two searches together (the question and a search text the model proposes); top 4 passages each.
- Answer only if the nearest passage's distance ≤ `CHAT_MAX_RELEVANCE_DISTANCE = 0.64`.
- `source_documents` names each standard, guide or policy once; empty if refused.
- Out-of-scope questions get `CHAT_REFUSAL_MESSAGE`.

> **Where it is used in StreetCare AI:** `answer_from_documents(collection, document_entries, question_text, conversation)`; every turn is stored with its closest distance.

INTERVIEW QUESTIONS

Q. How do you evaluate a RAG system?

A. Separately: retrieval (is the right document near the top?) and generation (faithful to passages, correct refusals on out-of-scope questions).

63 Reading rules from documents

RAG Data

Definition. Parsing requirements and their figures out of a standard at run time, instead of writing them in code.

HOW IT WORKS

- Check §2: `"The defect type is Carriageway."` must match the expected type, or refuse.
- Take §4's list; split by requirement number; read each name and sentence.
- Match the sentence against the ten `REQUIREMENT_SENTENCE_PATTERNS` to find its kind and capture its figures.
- Hold figures as Decimals under `minimum`, `maximum`, `footway_limit`, `carriageway_limit`.

EXAMPLE

```
"3 Depth. The defect has a depth or a vertical step of at least 40 millimetres."
→ {"kind": "defect_depth", "rules": {"minimum": Decimal("40")}}
```

Where it is used in StreetCare AI: `knowledge/standard_reading.read_defect_standard(defect_type, document_entries)`.

INTERVIEW QUESTIONS

Q. What if a standard is edited?

A. Re-index, retrain (requirement counts change) and restart. No code changes, because no figure lives in code.

PART 8

Agents, LangGraph & human-in-the-loop

A ten-node workflow that stops for an inspector and survives a restart.

64 Agents as single LLM calls

Agents GenAI

Definition. An **agent** here is one LLM call with a narrow job and a fixed output layout, not a free-roaming loop. Tools do the work; agents interpret it.

EXAMPLE

Agent	Job	Returns
review_planner_agent	what the review must establish	PRIORITY · CHECKS · RATIONALE
requirements_analyst_agent	explain the screening + triggers	5 fields
evidence_analyst_agent	review documents with the guides' checks	4 fields
risk_analyst_agent	explain the two figures	3 fields
review_writer_agent	recommend a decision	RECOMMENDATION · RECOMMENDED DECISION
order_writer_agent	works order or inspection request	SUBJECT · DOCUMENT
resident_update_writer_agent	resident update after an order	HEADLINE · UPDATE

Where it is used in StreetCare AI: `agentic/agents.py`.

INTERVIEW QUESTIONS

Q. Why one call per agent instead of a tool-calling loop?

A. Predictable cost (exactly 5/6/7 calls), easy auditing, and no chance for a model to invoke an action on its own.

65 LangGraph: state, nodes, edges & reducers

Agents

Definition. LangGraph builds a workflow as a graph: **nodes** are functions, **edges** decide what runs next, and a typed **state** dictionary flows through.

HOW IT WORKS

- A node takes the state and returns only the keys it changed.
- A **reducer** combines updates: `llm_call_count` is added to; every other key is replaced.
- Ten nodes registered under exact names; the first seven run in order from START.
- Building the graph runs nothing and calls no model.

EXAMPLE

```
builder = StateGraph(ReportReviewState)
builder.add_node("load_report_context", load_report_context_node)
...
builder.add_edge(START, "load_report_context")
builder.add_edge("load_report_context", "plan_report_review")
builder.add_edge("record_report_outcome", END)
```

Where it is used in StreetCare AI: `agentic/state.py`, `nodes.py`, `graph.build_workflow(checkpoint_path)`.

INTERVIEW QUESTIONS

Q. LangGraph vs a simple chain?

A. Chains are linear. Graphs allow branches, pauses for humans and resuming from saved state.

66 Conditional routing

Agents

> **Definition.** Choosing the next node from the state at run time, through a router function.

HOW IT WORKS

- Exactly two conditional branches, both chosen by `route_report_decision`, which returns the decision itself.
- Out of the gate: ORDER/INSPECT → `draft_order_document`; REFER → `record_report_outcome`.
- Out of `draft_order_document`: ORDER → `draft_resident_update`; INSPECT → `record_report_outcome`.
- Every other connection is a plain edge.

EXAMPLE

```
builder.add_conditional_edges("await_inspector_decision", route_report_decision,
    {DECISION_ORDER: "draft_order_document", DECISION_INSPECT: "draft_order_document",
     DECISION_REFER: "record_report_outcome"})
```

> **Where it is used in StreetCare AI:** `agentic/graph.py`.

INTERVIEW QUESTIONS

Q. Why route on the decision name itself?

A. No hidden mapping to get wrong: the path is literally named by what the inspector chose.

67 Human-in-the-loop with interrupt

Agents

> **Definition.** Pausing a workflow so a person can make the decision, then continuing with that decision.

HOW IT WORKS

- `await_inspector_decision_node` calls `interrupt(...)`, handing out the review_id.
- The API returns 201 with the review AWAITING_DECISION.
- Later, `POST /reviews/{id}/decision` resumes with `Command(resume=decision)` on the same thread_id.
- The paused node runs again from its start on resume, so it writes nothing before the interrupt.

> **Where it is used in StreetCare AI:** `agentic/nodes.py` node 7; `start_report_review` and `submit_review_decision` in the API.

INTERVIEW QUESTIONS

Q. Why must the paused node write nothing?

A. Because it re-runs from the top on resume; any write before the interrupt would happen twice.

68 Checkpointing & resuming after a restart

Agents

> **Definition.** Saving the graph's state after every node to durable storage, so a paused run can be resumed later, even by another process after the machine restarts.

HOW IT WORKS

- Compiled with a SQLite checkpointer writing to `database/checkpoint.db`.
- The connection must be usable from more than one thread (`check_same_thread=False`), because FastAPI serves from a thread pool.
- Earlier nodes are not re-run on resume, so their LLM calls are not paid for twice.

EXAMPLE

```
conn = sqlite3.connect(checkpoint_path, check_same_thread=False)
workflow = builder.compile(checkpointer=SqliteSaver(conn))
```

> **Where it is used in StreetCare AI:** `build_workflow(checkpoint_path)`; `CHECKPOINT_DATABASE_FILE`.

INTERVIEW QUESTIONS

Q. Why keep streetcare.db and checkpoint.db together in backups?

A. The review row says AWAITING_DECISION and the checkpoint holds the paused run. Restoring one without the other leaves a review that can't be resumed.

69 Idempotent resume & failure recovery

Agents API

> **Definition.** Designing a resume so sending the same request again after a failure finishes the job instead of doing it twice.

HOW IT WORKS

- Run status: RUNNING → AWAITING_DECISION → COMPLETE, or FAILED.
- If node 8–10 fails, the transaction rolls back; the checkpoint still holds the decision.
- The same decision sent again resumes and finishes; a different decision gets 409.

> **Where it is used in StreetCare AI:** `find_review_conflict`; `record_report_outcome_node`.

INTERVIEW QUESTIONS

Q. Why does a different decision get a conflict?

A. The checkpoint has already accepted one decision; switching mid-way would leave half-written documents for the wrong outcome.

PART 9

Testing & engineering practice

How the project proves it works, and the habits that keep it honest.

70 Automated testing with pytest

Testing Python

Definition. Code that checks your code, so every change is verified in seconds.

HOW IT WORKS

- **Unit tests**: one function with small inputs (4.5 cm → 45 mm; the §13 worked example → INSPECT ON SITE).
- **Integration tests**: FastAPI `TestClient` calls real routes with seeded users.
- **Fixtures** like `tmp_path` give each test its own database and checkpoint file.
- **Fakes**: replace the Gemini call with one that returns fixed labelled fields: free, fast, deterministic.

EXAMPLE

```
def test_inch_reading_rounds_half_up():
    frame = pd.DataFrame({"defect_depth_mm": ["1.75 in"]})
    assert convert_readings(frame, config.FORM_READINGS).loc[0, "defect_depth_mm"] == 44

def test_adviser_cannot_read_queue(client, adviser_token):
    assert client.get("/queue", headers={"X-Session-Token": adviser_token}).status_code == 403
```

Where it is used in StreetCare AI: `tests.py` (to be written); the expected-output figures in the specification are natural acceptance tests.

INTERVIEW QUESTIONS

Q. Why is hard-coding to pass tests dangerous?

A. Tests sample behaviour. Code that special-cases the sampled inputs passes and fails on every real input. The marking suite will use inputs you haven't seen.

71 Secrets, configuration & reproducibility

Security Python

Definition. Keeping settings in one place, secrets out of code and git, and results identical from run to run.

HOW IT WORKS

- Secrets in `.env` via python-dotenv; `.env` in `.gitignore`.
- CI tokens in repository secrets, never in workflow files.
- Fixed seed 1313 for splits and models; JSON written with sorted keys.
- Generated folders (`processed/`, `output/`, `database/`) rebuilt by the entry points, never committed.

> **Where it is used in StreetCare AI:** `ENV_FILE`, `GEMINI_API_KEY_VARIABLE`, `GEMINI_MODEL_VARIABLE`, `RANDOM_SEED`.

INTERVIEW QUESTIONS

Q. What would you do about the key already committed in this repository?

A. Rotate it immediately, remove `.env` from git (and its history if shared), add it to `.gitignore`, and do the same for the Sonar token in `build.yml`.

EXTRAS

Rapid-fire interview bank

Short answers to say out loud. Each one is grounded in the StreetCare design.

Q. Walk me through what happens when a review is opened.

A. The API checks the token and the review permission, takes the report's lock, and starts the LangGraph run. Node 1 gathers every fact with tools (screening, permits, triggers, evidence, flags, predictions). Five agents explain them in one call each, the review is stored AWAITING_DECISION in one transaction, the run pauses at the gate and is checkpointed, and the API answers 201.

Q. Why are no requirement limits in the code?

A. The specification makes the defect standard PDFs the only source of rules. Reading them at run time means the near-identical carriageway and footway standards can never be swapped, and a standard can change without a code change.

Q. How did you handle the messy data?

A. Read everything as text, map null tokens, then a fixed order of repairs: duplicates, identifiers, references, spellings, whole numbers, readings, impossible values, latest revision, missing target. Anything not in the rules is refused with an error naming the column and values. History train goes 32,738 → 30,569.

Q. Why Decimal instead of float?

A. Limits are met or missed by the last digit. Decimal plus one half-up rounding gives 1.25 in = 32 mm every time, exactly as the Report Rules' worked example.

Q. How is the assessment decided?

A. Any requirement Not met or Not documented → INSPECT ON SITE; otherwise any coordination trigger → REFER FOR COORDINATION; otherwise ORDER REPAIR. Requirements first.

Q. What are the coordination triggers?

A. A live permit on the street in the same part of the highway; a traffic-sensitive street for carriageway or drainage defects; a lighting column taller than 8 m. Blanks count as triggering.

Q. What does the repeat model learn from?

A. The report history at its latest revisions, with requirement counts from screening each past report under its standards, plus trigger flags. It is a logistic regression that must reach a held-out ROC AUC of 0.80 and react the right way to an extra unmet requirement or permit.

Q. Why a random forest for crew hours?

A. Repair time depends on interactions (method × traffic management × street). A forest of 300 trees with leaves of at least 10 captures them without memorising; it must reach R^2 0.86.

Q. How is leakage prevented?

A. Separate train and eval files; preparation inside the pipeline fitted on train only; only features known when a report arrives.

Q. What does the sentence model do?

A. TF-IDF + logistic regression labels each sentence with one of nine types; the type says which requirements it bears on. Only Measurement sentences are searched for figures, which are compared exactly with the report and flagged.

Q. How is the knowledge base built?

A. pypdf text, split into numbered sections (136 across 16 documents), 700-character passages with 120 overlap, embedded with all-MiniLM-L6-v2 into Chroma using cosine distance.

Q. Why look sections up by number instead of searching?

A. The standards share one template; similarity could return another type's identical-looking section. A metadata lookup can't.

Q. Describe the review graph.

A. Ten nodes: load context, plan, assess requirements, analyse statements, interpret predictions, prepare recommendation, await decision (interrupt), draft order document, draft resident update, record outcome. Two conditional branches chosen by route_report_decision.

Q. How many LLM calls does a review cost?

A. Five before the gate; seven in total for ORDER, six for INSPECT, five for REFER. Every accepted call is stored with tokens and latency.

Q. How does the human gate work?

A. interrupt() pauses the run; the SQLite checkpointer saves it. The decision route takes the report lock, runs the conflict and refusal checks (inspector, district, not the opener, decision still open), then resumes with the decision on the same thread.

Q. What survives a restart?

A. Paused reviews: the checkpoint is on disk and the connection works across threads, so a new process can resume them.

Q. How do you stop the LLM inventing figures?

A. It is only given facts tools computed, it returns labelled fields that are parsed strictly, and outgoing documents may not contain model figures or template slots.

Q. How are passwords stored?

A. PBKDF2-HMAC-SHA256, 200,000 rounds, a random 16-byte salt per account; compared in constant time.

Q. How is access enforced?

A. Every protected route is on a router guarded by authorize_request: the session token finds the user, PERMISSION_BY_ROUTE gives the permission, ROLE_PERMISSIONS must include it. The dashboard only hides pages for convenience.

Q. What can a customer services adviser learn?

A. A report's types, street and status, and once ordered where the defect is and what the resident was told. Never a prediction; never a street figure from fewer than five reports.

Q. How does the chat avoid answering from nowhere?

A. It answers document questions only if the nearest passage is within cosine distance 0.64, lists its sources, excludes the casebook, and refuses anything else.

Q. How would you test the LLM parts without spending money?

A. Replace the Gemini call with a fake that returns fixed labelled fields; test parsing, refusals, call counts and routing deterministically.

Q. What would you improve next?

A. Probability calibration for the repeat model, SHAP explanations on the detail page, async LLM calls to cut review latency, hybrid keyword + vector retrieval for the chat, and drift monitoring on the live queue.

Explain the project: 30 seconds & 2 minutes

30-SECOND VERSION

"StreetCare AI is a street-defect review platform for a borough council. It cleans reports from three channels into exact readings, screens each one against the requirements it reads from the council's own PDF standards, checks the street for works and other coordination triggers, reads every sentence of the attached notes with an NLP model, and estimates repeat likelihood and crew hours. A ten-node LangGraph workflow has five Gemini agents explain all of that and recommend a decision, then stops until a Highways Inspector from that district decides. The pause survives restarts. Everything is served through a role-secured FastAPI service and a Streamlit dashboard, and fully audited."

2-MINUTE VERSION: TALK THROUGH IT IN THIS ORDER

1. **Problem**: three channels write reports differently; two standards look alike but differ; notes contradict reports; street works need coordinating; residents' privacy.
2. **Data**: text-first loading, null tokens, canonical mappings, exact Decimal unit conversion, latest revision, refusal instead of guessing, quality report (history 32,738 → 30,569).
3. **Knowledge**: 16 PDFs → 136 sections → 700-char passages → MiniLM embeddings in Chroma; each standard's requirements parsed with their figures at run time.
4. **Screening**: Met / Not met / Not documented per requirement, three triggers, assessment by fixed precedence, decisions left open.
5. **ML**: logistic repeat model with requirement counts and trigger flags (AUC ≥ 0.80), random-forest crew hours (R^2 ≥ 0.86), TF-IDF sentence model (macro F1 ≥ 0.95); behavioural checks; gated before saving.
6. **Agents**: ten-node LangGraph, one Gemini call per agent, labelled fields parsed strictly, 5 calls before the gate and 7 / 6 / 5 in total.
7. **Human gate**: interrupt + SQLite checkpoint; per-report lock; inspector, district and not-the-opener checks before resuming.
8. **Service & safety**: PBKDF2 passwords, session tokens, RBAC on one guarded router, six audit tables, a chat that answers only from documents and never gives advisers predictions or small-group figures.

Numbers to remember

Fact	Value
Council	48 streets · 4 districts · 3 channels · 5 defect types
Training rows received → kept	history 32,738 → 30,569 · 8,202 → 7,648
	repair logs 26,095 → 25,537 · 6,476 → 6,337 · sentences 36,661 → 35,717
Live files	45 rows → 40 open reports · 62 documents · 29 rows → 28 permits (23 live)
Knowledge base	16 PDFs (5 standards, 8 guides, policy, rules, casebook) · 136 sections
Chunks & embeddings	700 characters · 120 overlap · all-MiniLM-L6-v2 (384-d) · cosine

Model minimums	repeat ROC AUC 0.80 · crew hours R^2 0.86 · sentence macro F1 0.95
Model settings	logistic 3,000 iterations · forest 300 trees, leaf 10, max features 0.7 · seed 1313
Sentence hold-out	20% of labelled sentences · 9 types
Likelihood bands	HIGH ≥ 0.40 · MEDIUM ≥ 0.20 · LOW below · rounded to 0.01
Crew hours	rounded to 0.1 · overrun if > 1.3 × planned
Triggers	live works · traffic-sensitive (carriageway, drainage) · column > 8 m
Review graph	10 nodes · 7 agents · 2 conditional branches · 5 calls before the gate · 7 / 6 / 5 total
Storage	6 tables · report_reviews has 37 columns · checkpoint.db for paused runs
Accounts	16 seed accounts · 3 roles · 2 Highways Inspectors per district
Security	PBKDF2-SHA256 · 200,000 rounds · 16-byte salt · 24-byte session token
Chat	7 question types · top 4 passages · distance ≤ 0.64 · last 3 turns · group size ≥ 5
Service	API :8081 · dashboard :8080 · 300 s client timeout · 11 routes